

Session 3 · Deck 3.B4

The Data Workflow: Acquire, Clean, Persist, Export

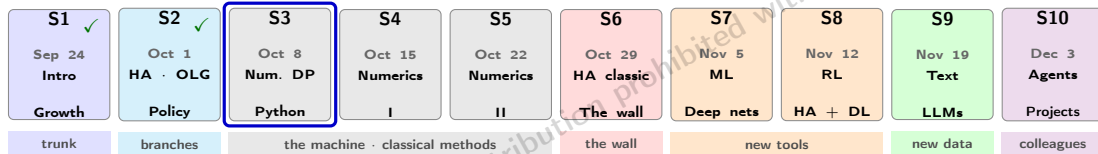
Zhigang Feng

Thursday, October 8, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 3.B4 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 3.B1 your toolkit · 3.B2 from loops to arrays · 3.B3 PyTorch, and coding with AI · 3.B4 the data workflow.

Your Model Produces Moments. So Does the Data.

Everything this morning ended in a number the *model* produced. Calibration and estimation are the comparison of those numbers with the ones the **data** produces — and that comparison is not free. It needs the series pulled, the units fixed, the trend removed, and the result saved somewhere you can find it again.

From the model	simulate, HP-filter, take σ and ρ
From the data	download, clean, HP-filter, take σ and ρ

The two halves must match

The same filter, the same frequency, the same detrending. A business-cycle statistic compared against one computed a different way is not a comparison at all.

Homework 1, Part B asks for exactly this: simulate the stochastic growth model, HP-filter the simulated series, and report the moments. This deck is the other half of that instruction.

The Data Workflow

The pipeline in an empirical macro project:

1. **Acquire** the data — FRED, the World Bank, a CSV from a coauthor.
2. **Clean** — missing values, types, dates.
3. **Persist** — save the clean version, so you never redo steps 1–2.
4. **Analyze and visualize** — moments, regressions, plots.
5. **Export** — figures and tables for the paper.

Step 3 is the one people skip

Re-downloading on every run makes your results depend on a web service and on the day you ran it. Save the clean data once; commit the *script* that produced it.

Vintages change. FRED revises GDP. A figure you cannot reproduce next March is a figure you cannot defend in a referee report.

Acquire: FRED GDP Data

```
import pandas_datareader.data as web
import datetime

start = datetime.datetime(1990, 1, 1)
end    = datetime.datetime(2025, 1, 1)

# 'GDP' is the FRED series ID
# for nominal GDP (quarterly)
gdp = web.DataReader('GDP', 'fred',
                     start, end)

print(gdp.head())
```

- pandas_datareader sends a request to the FRED API.
- FRED identifies every series by a **series ID** — GDP, GDPC1 (real), UNRATE, FEDFUNDS.
- What comes back is a **pandas DataFrame**: a table with a date index.
- It needs an internet connection. Many simple calls work without a key; heavy use wants one.

GDP is *nominal*. For business cycles you almost always want GDPC1.

Clean: Growth Rates and Missing Values

```
import pandas as pd
import numpy as np

# quarterly growth rate
gdp['growth'] = gdp['GDP'].pct_change()

# the first difference is NaN
clean = gdp.dropna()

# a subsample
post_2000 = clean[clean.index >= '2000-01-01']

print(post_2000[['GDP', 'growth']].head())
```

QM Lec. 3, "Clean: Compute Growth and Filter".

A DataFrame column behaves like a NumPy array with a label and a date index attached, so `pct_change` and `dropna` are vectorised for the same reason the Bellman update was.

`dropna()` is silent. **Print how many rows it removed.** A filter that quietly deletes a third of your sample is the most expensive kind of bug, because the code still runs and the plot still looks fine.

The HP Filter: Trend and Cycle

```
import numpy as np
from statsmodels.tsa.filters.hp_filter \
    import hpfilter

gdp_log = np.log(df['GDPC1'])

# lambda = 1600 for quarterly data
cycle, trend = hpfilter(gdp_log, lamb=1600)

df['gdp_trend'] = trend
df['gdp_cycle'] = cycle

# the business-cycle moments
print(df['gdp_cycle'].std())
print(df['gdp_cycle'].autocorr(1))
```

The filter splits a series into a slow **trend** and a stationary **cycle**. $\lambda = 1600$ is the quarterly convention; 100 for annual, 129,600 for monthly.

The last two lines are the object of comparison: the standard deviation and the first-order autocorrelation of the cyclical component. Your model must produce the same two numbers, computed the same way.

It is a convention, not a theory — it invents cycles at the ends of the sample and its filtered series can mislead (Hamilton 2018). Report what you used.

QM Lec. 3, "Example: HP Filter (Trend and Cycle)". The moment lines and the caveat are added here, because HW1 Part B needs them.

Persist: Save the Clean Version

```
# CSV: shareable, readable by anything
post_2000.to_csv("gdp_post2000.csv")

# pickle: Python-only, preserves dtypes
post_2000.to_pickle("gdp_post2000.pkl")

# later, in a different script
df = pd.read_csv("gdp_post2000.csv",
                 index_col='DATE',
                 parse_dates=True)
```

- **CSV** for anything a coauthor, a referee or your future self has to open.
- **Pickle** for speed and exact types, inside one project only — it is not a durable format.
- `parse_dates=True` is what turns the index back into dates rather than strings.

This is the same discipline as 3.A1's number ledger: the computation writes a file, and everything downstream reads that file instead of recomputing.

Visualize: GDP Growth

```
import matplotlib.pyplot as plt

x = df.index
y = df['growth'] * 100          # percent

plt.figure(figsize=(10, 5))
plt.plot(x, y, label="quarterly growth (%)")
plt.axhline(0, linestyle='--', color='grey')
plt.title("US Real GDP Growth")
plt.xlabel("year")
plt.ylabel("growth (%)")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
```

Nothing here is specific to economics — it is the same matplotlib you used to plot a policy function this morning, pointed at a date index.

Label the axes and state the units on every figure you make, including the throwaway ones. The throwaway figure is the one that ends up in a seminar.

QM Lec. 3, "Visualize: GDP Growth". The source sets a seaborn theme; dropped here so the deck needs only the packages in labs/requirements.txt.

Export: Figures for the Paper

```
# raster, for slides
plt.savefig("gdp_growth.png", dpi=300,
            bbox_inches='tight')
```

```
# vector, for LaTeX
plt.savefig("gdp_growth.pdf",
            bbox_inches='tight')
```

```
plt.show()
```

savefig must come **before** show() — showing a figure clears it, and you get an empty file.

QM Lec. 3, "Export: High-Quality Figures".

Vector for a paper, always

A PDF stays sharp at any zoom and its text is real text. A 300-dpi PNG is for slides and for anything a journal insists on rasterising.

Every figure in Session 3's decks was produced this way — a script wrote the numbers to a file, and the figure was drawn from that file, never by hand.

Takeaways

- Calibration is a **comparison**, and a comparison is only valid if both sides were computed the same way — same filter, same frequency, same detrending.
- The pipeline is **acquire** → **clean** → **persist** → **analyze** → **export**, and step 3 is the one that makes the other four reproducible.
- `dropna()` is silent. Print what it removed, every time.
- The **HP filter** with $\lambda = 1600$ is the quarterly convention, and the two numbers that come out of it — the standard deviation and the autocorrelation of the cycle — are what your model has to match.
- It is a convention, not a theory. Say what you used, and save the file that produced it.

Bridge: What You Owe, and What Comes Next

Session 3 in one sentence. A model on paper becomes a number through six decisions; we made all six badly, measured exactly how badly, and named the technique that repairs each one.

- **Lab L03** repeats Block A end to end: the `GrowthModel` class, VFI, time iteration, and the closed-form check.
- **Homework 1, Part A** is out today, due October 22. It begins exactly where 1.B1 said it would — at the Brock–Mirman benchmark.

Next week (S4)

Numerical Methods I: root-finding and optimization — we open the `brentq` box and retire the grid search — then differentiation, approximation, quadrature, and where Π comes from.

Before then: install the environment and run L03 end to end. Judd (1998), ch. 12, for the reference treatment.